

Assignment7 - Due: Tuesday, November 16th, 5:30pm

CSE310 Assignment 7

Due Date

Due: Tuesday, November 16th, 5:30pm

Important: This is an individual assignment. Please do not collaborate. Do not share your solution with anyone.

Make sure to follow the academic integrity policies.

Make sure that you write every line of your code. Using code written by someone else will be considered a violation of the academic integrity and will result in a report to the Dean's office. Your code should not match with anything found online.

Copying any part of this assignment, and providing them to another person or posting them on the Internet without a permission of the instructor will be a violation of its copyright. <http://www.asu.edu/copyright/> (<http://www.asu.edu/copyright/>).

It must be submitted on-line (course website)

Go to "GradeScope" tab on Canvas -> CSE310 -> Assignment7, and upload your program files including Makefile.

No late assignment will be accepted.

Files To be Submitted:

You are required to turn in the following source file(s) in C++:

Assignment7.cpp

LinkedList.h

Graph.h

Stack.h

Makefile

README.txt

You can also submit additional files

Objectives:

-This is to practice making graphs and perform some operations.

Assignment Description:

This assignment is to implement a graph data structure using an adjacency list and also to find a circuit (a sequence of edges starting from one node/vertex and ending to itself) using every edge of the graph.

1. Assignment7.cpp file (a driver program) needs to be created to process the information from the file named "graph.txt". The file "graph.txt" will have some information on an undirected graph, and its first line has the format of:

n m

where n is the number of nodes/vertices and m is the number of edges in the graph. Then the nodes/vertices of this graph are named as 1, 2, ... n. A real example for this can be:

12 35

In this case, there will be 12 nodes/vertices and 35 edges. After this line, there will be m lines containing the information of edges in the format of:

s e w

where s and e are two nodes/vertices of the edge and w is the weight of the edge. Each node can have a weight. A real example for this can be:

1 2 5

In this case, there is an edge between the vertex 1 and the vertex 2, with its weight of 5.

Here is a sample graph.txt file:

graph.txt (<https://canvas.asu.edu/courses/95251/files/39970116?wrap=1>) 
(https://canvas.asu.edu/courses/95251/files/39970116/download?download_frd=1)

2. In this assignment, you need to implement an adjacency list that is an array of linked lists. First you need to create a LinkedList struct/class whose each node containing an adjacency vertex, its weight, and also a variable "notUsed" that will be used in the later step of the assignment. You also need to define insertEdge() function in the LinkedList in order to add each edge information. You need to add each edge information in the increasing order of the adjacent vertices. In this assignment, it is possible to have more than one edge between two nodes, and in that case, add them in the increasing order of their weights. For example, if we have the following three edges:

1 2 1

1 3 1

1 2 2

Then the linked list of the vertex 1 needs to have its adjacent vertices in the following order:

head -> vertex 2, weight 1 -> vertex 2, weight 2 -> vertex 3, weight 1

Because this is an undirected graph, if you see the edge information 1 2 1, then you need to store the information of two edges, an edge from 1 to 2 with its weight 1 and also an edge from 2 to 1 with its weight 1.

After creating the Linked List data structure, you need to create Graph.h and create an adjacency list that is an array of linked lists in it. The size of the array is the number of nodes/vertices, and each slot should correspond to each vertex in their increasing order. You also need to compute and record the degree (the number of out-going edges) for each vertex. After populating the adjacency list, you will need to define the printGraph() function to print out its content using the following format:

The Graph contains:

From: 1 with its degree 4

To: 2 with weight 1

To: 2 with weight 2

To: 3 with weight 1

To: 4 with weight 1

From: 2 with its degree 2

To: 1 with weight 1

To: 1 with weight 2

3. In this step, you will compute its circuit (a sequence of edges to go through every edge of the graph, starting from one node/vertex and ending to itself). First you need to define the Stack data structure by creating a struct/class of Edge containing two vertices and its weight and making an array of edges or a linked list of edges (either of them can represent a stack as long as we use LIFO, Last-In-First-Out order). You also need to define the push() function to push an edge and the pop() function to pop the last edge. Create two variables "circuit" and "tempStack" of this Stack data structure, initially being empty.

Then you need to define the modified DFS (Depth First Search) and DFS_VISIT in the Graph.h in order to compute its circuit. First you always need to start the search from the vertex 1. The DFS and DFS_VISIT that we saw in class was calling DFS_VISIT for the vertices with white color, but here we call it for the vertices with white or gray color. The aim is to explore every edge that is incident with each vertex. Once we explore an edge by visiting an adjacent vertex, you need to mark the edge as "used" (see the step 2 for this) and push the edge onto the "tempStack". We mark such edge as "used" in the adjacency list so that we will not explore it again later on. Remember that each edge is stored twice in the adjacency list because it is an undirected edge. If a vertex does not have any more edge that is incident with it, then pop the last edge from the "tempStack", say (u, v), then switch the order of its vertices to (v, u) and push/append it to the "circuit". At this point, the search will need to back-track to the vertex u.

To show its intermediate stages, every time “tempStack” and “circuit” change (pop() or push() is called), you need to print out their content using the following format:

Circuit: (1,3)

Stack: (1,2) (2,3)

Circuit: Empty

Stack: (1,2) (2,3)

Once the DFS is completed, your program needs to print out the edges in the circuit using the following format:

The circuit is:

(1,3)

(3,6)

(6,7)

...

(2,1)

Note that your program can be tested with different inputs from the above sample input, but the format will be the same.

4. You must submit a file called “Makefile” that compiles your Assignment7.cpp, LinkedList.cpp, Graph.h, Stack.h files and generate an executable file called “assignment7” to the gradescope. Make sure that your program is generating correct outputs when you submit your files.

The following is a partial Makefile. **Makefile**

(<https://canvas.asu.edu/courses/95251/files/39970131?wrap=1>) 

(https://canvas.asu.edu/courses/95251/files/39970131/download?download_frd=1)

Once you create this file, then you can type "make" in a command line to compile and it should generate an executable file called "assignment7".

```
-----  
EXEC = assignment7  
  
CPP = g++  
  
CFLAGS = -c -Wall  
  
$(EXEC)      : Assignment7.o Graph.o  
              $(CPP) -o $(EXEC) Assignment7.o Graph.o  
  
Assignment7.o : Assignment7.cpp  
              $(CPP) $(CFLAGS) Assignment7.cpp  
  
Graph.o       : Graph.h Graph.cpp  
              $(CPP) $(CFLAGS) Graph.cpp  
  
clean  :  
        rm *.o  
-----
```

To compile type (in a command line such as linux, ubuntu, mac terminal):

make

To execute:

./assignment7

You need to have the "graph.txt" file in the same folder as
Makefile/Assignment7.cpp files

To store its output into a file named myout1.txt:

./assignment7 > myout1.txt

Then you can compare myout1.txt and output1.txt (its expected output given
below)

[output1.txt \(https://canvas.asu.edu/courses/95251/files/39970163?wrap=1\)](https://canvas.asu.edu/courses/95251/files/39970163?wrap=1) 
([https://canvas.asu.edu/courses/95251/files/39970163/download?
download_frd=1](https://canvas.asu.edu/courses/95251/files/39970163/download?download_frd=1))

Implementation/Documentation Requirements:

- You need implement this program using C++ and it has to read from a file (graph.txt), and prints its output to a screen (the standard output).
 - Your program needs to compile and execute in general.asu.edu.
 - You need to define Graph class with insertEdge() , printGraph(), DFS(), DFS_VISIT(). You can decide which parameters are needed. Additional functions can be added.
 - You need to define LinkedList class/struct with insertEdge(). This is to specify an adjacent node and its weight to be added. This function should be called by insertEdge() in the Graph that specify two vertices, and its weight. You can decide which parameters are needed.
 - You need to define Stack class/struct with pop() and push() functions.
 - Your code should be well documented and commented.
 - You must use the provided data sets.
 - Also you are not allowed to use any predefined data structures (such as ones in the library in C++, etc.) except arrays and strings (you can use predefined functions of the string), you need to build your own data structures and operations associated with them (such as insert or search).
 - Copying any codes from other people's programs or online is considered to be cheating and will lead to a report to the Dean and you will be penalized. If the code you submitted is not your original work, it can be considered as an academic integrity policy violation. Also check the rules stated in the syllabus of the course regarding the academic integrity policies.
-

What to turn in:

You must turn in C++ source code using the Gradescope.

The types of files that you can submit are .cpp, or .h files. Your program needs to compile and execute correctly in the gradescope.

Your program should be well commented and documented, make sure the first few lines of your program contain your name, your ASU email address, and a description of each file. You should use indentation, good variable names, and clear, easy to read, and specific comments. (You will be graded on comments.)

Input

Here is a sample content of graph.txt file

graph.txt (<https://canvas.asu.edu/courses/95251/files/39970116?wrap=1>) 
([https://canvas.asu.edu/courses/95251/files/39970116/download?
download_frd=1](https://canvas.asu.edu/courses/95251/files/39970116/download?download_frd=1))

Output

Here is the output for the above graph.txt file

(<https://canvas.asu.edu/courses/95251/files/36961650?wrap=1>) **output1.txt**
(<https://canvas.asu.edu/courses/95251/files/39970163?wrap=1>) 
([https://canvas.asu.edu/courses/95251/files/39970163/download?
download_frd=1](https://canvas.asu.edu/courses/95251/files/39970163/download?download_frd=1))

Your program will be tested with more test cases other than these test cases.

Error Handling

Your program will be tested with other input files besides the one above, thus is expected to be robust. Your program might also be tested using test cases not given to you.

Grading Criteria

____/ 1 Makefile is submitted and it works correctly. Your Makefile should compile your source code and should create an executable file name "assignment7". If this part does not work, then you will get zero on this assignment

____/ 3 Documentation (correct and complete code description and comments, header for each function/method, each file should have your name at its top) and good indentation and spacing (easy to read) A

README file needs to be provided to describe your project, what information is in each file

____/ 1 Assignemnt7cpp file is provided to handle the file read.

____/ 1 the Graph data structure (class) is defined using an adjacency list correctly and it has a description and is well commented

____/ 1 the printGraph function for the graph is defined correctly (and it produces the correct output) and it has a description and is well commented

____/ 3 the DFS and DFS_VISIT functions for the graph are defined correctly (and it produces the correct output) and it has a description and is well commented

____/ 2 the insertEdge function for the linked list and the insertEdge function for the graph are defined correctly (and it produces the correct output) and it has a description and is well commented

____/ 2 the push and pop functions for the stack are defined correctly (and it produces the correct output) and it has a description and is well commented

____/ 16 correct outputs including correct vertices, edges, degrees, correct graph content, intermediate circuit/stack contents, and the final circuit result for all outputs.

Total points: 30

*Copyright © 2021,
Arizona State University
All rights reserved.*

ASU disclaimer [_\(http://www.asu.edu/asuweb/disclaimer/\)](http://www.asu.edu/asuweb/disclaimer/).